

C++ & Backend Development Interview Mastery Guide

150+ Questions & Answers | Product-Based Companies | FAANG-Level

Section	Topics	Questions
1	C++ Fundamentals	Q1 – Q20
2	OOP in C++	Q21 – Q40
3	Memory & Pointers	Q41 – Q55
4	STL & Templates	Q56 – Q70
5	Concurrency & Multithreading	Q71 – Q85
6	Modern C++ (C++11/14/17/20)	Q86 – Q100
7	Backend Development Fundamentals	Q101 – Q120
8	System Design & Architecture	Q121 – Q135
9	Product-Based Company Special	Q136 – Q155

SECTION 1: C++ Fundamentals

Q1. What are the key differences between C and C++?

[Core]

C++ is a superset of C that adds object-oriented programming, templates, exceptions, the Standard Template Library, and stronger type checking. C is procedural; C++ supports procedural, OOP, and generic programming paradigms.

Q2. What is a namespace and why is it used?

[Core]

A namespace groups related identifiers to avoid name collisions. 'std' is the standard library namespace. You use 'using namespace std;' or qualify explicitly with 'std::'. Namespaces can be nested and aliased.

```
namespace MyLib { int value = 10; }  
int x = MyLib::value;
```

Q3. Explain the difference between #define and const.

[Core]

#define is a preprocessor macro — it does simple text substitution with no type safety. const is a typed, scoped constant processed by the compiler. Prefer const (or constexpr) over #define for constants.

Q4. What is the difference between 'struct' and 'class' in C++?

[OOP]

In C++, the only difference is default access: struct members are public by default; class members are private. Both support inheritance, constructors, and member functions. Conventionally, struct is used for POD types and class for objects with encapsulated behavior.

Q5. What is the scope resolution operator (::) used for?

[Core]

It accesses class members, namespace members, and global variables. It distinguishes between a local variable and a global one, or between a base class and derived class method with the same name.

```
int x = 5;  
void f() { int x = 10; std::cout << ::x; } // prints 5
```

Q6. What is function overloading?

[Core]

Function overloading allows multiple functions with the same name but different parameter lists (number, type, or order). The compiler selects the correct version at compile time. Return type alone is not sufficient to overload.

Q7. What is the difference between pass-by-value and pass-by-reference?

[Core]

Pass-by-value copies the argument — changes inside the function don't affect the original. Pass-by-reference (using &) passes the actual variable — changes affect the original. References avoid the cost of copying large objects.

```
void byVal(int x) { x = 10; } // original unchanged  
void byRef(int& x) { x = 10; } // original changed
```

Q8. What are inline functions?

[Core]

An inline function is a hint to the compiler to replace the function call with the function body at the call site, avoiding call overhead. The compiler may ignore the hint. Defined with the 'inline' keyword or implicitly when defined inside a class.

Q9. What is a reference variable? How is it different from a pointer?

[Core]

A reference is an alias for another variable. It must be initialised when declared and cannot be reassigned. It has no null state and no address arithmetic. A pointer can be null, reassigned, and used with arithmetic. References are safer; pointers are more flexible.

Q10. Explain the 'const' keyword in different contexts.

[Core]

'const int x' — constant value. 'const int* p' — pointer to const int (value cannot change). 'int* const p' — const pointer (address cannot change). 'const int* const p' — both. In methods, 'void f() const' means the method won't modify the object.

Q11. What is the difference between 'new' and 'malloc'?

[Memory]

'new' allocates memory and calls the constructor; 'delete' calls the destructor and frees memory. 'malloc' only allocates raw memory; 'free' only frees it — no constructor/destructor. 'new' throws std::bad_alloc on failure; malloc returns nullptr.

```
int* p1 = new int(5); // constructor called
int* p2 = (int*)malloc(sizeof(int)); // no constructor
```

Q12. What is a storage class in C++?

[Core]

Storage classes define the lifetime and visibility of variables: 'auto' (default, local scope), 'register' (hint for CPU register, deprecated in C++17), 'static' (persists across function calls), 'extern' (external linkage), 'mutable' (allows modification in const methods).

Q13. What is the difference between ++i and i++?

[Core]

Both increment i. Pre-increment (++i) increments first and returns the new value. Post-increment (i++) saves the old value, increments, and returns the old value. For complex types, pre-increment is generally more efficient.

Q14. What is type casting in C++?

[Core]

C++ provides four casts: static_cast (compile-time, safe conversions), dynamic_cast (runtime, polymorphic types), const_cast (add/remove const), reinterpret_cast (low-level bit reinterpretation). Prefer these over C-style casts for clarity and safety.

Q15. What is operator overloading?

[OOP]

Defining custom behaviour for operators (+, -, ==, <=, etc.) for user-defined types. Implemented as member or friend functions. Cannot overload ::, ., .*, ?:, sizeof. Overloading should preserve the intuitive meaning of the operator.

```
class Vec { int x,y;  
Vec operator+(const Vec& o) const { return {x+o.x, y+o.y}; }  
};
```

Q16. What are default arguments?

[Core]

Parameters with a default value that is used when the caller omits that argument. Defaults must appear from right to left in the parameter list. They are specified once — typically in the declaration (header), not the definition.

Q17. What is the difference between declaration and definition?

[Core]

A declaration introduces an identifier and its type. A definition also allocates storage or provides the function body. A variable/function can be declared many times but defined exactly once (One Definition Rule — ODR).

Q18. What is a header guard and why is it needed?

[Core]

A header guard prevents a header file from being included more than once in the same translation unit, avoiding redefinition errors. Use `#ifndef/#define/#endif` or the modern `'#pragma once'`.

```
#ifndef MY_HEADER_H  
#define MY_HEADER_H  
// declarations  
#endif
```

Q19. What is a translation unit?

[Core]

A translation unit is a single .cpp source file after preprocessing (includes expanded, macros replaced). The compiler processes one translation unit at a time. The linker combines all translation units into the final executable.

Q20. What is the difference between compile-time and run-time errors?

[Core]

Compile-time errors are caught by the compiler (syntax errors, type mismatches). Run-time errors occur during execution (null dereference, division by zero, out-of-bounds). Prefer moving checks to compile-time using `static_assert`, `constexpr`, and templates.

SECTION 2: Object-Oriented Programming in C++

Q21. What are the four pillars of OOP?

[OOP]

Encapsulation (bundling data and methods, hiding implementation), Abstraction (exposing only relevant interface), Inheritance (reusing and extending classes), Polymorphism (one interface, many forms — compile-time via overloading/templates and runtime via virtual functions).

Q22. What is a constructor and its types?

[OOP]

A constructor initialises an object. Types: Default (no args), Parameterised, Copy (takes const ref of same type), Move (takes rvalue ref, C++11), Delegating (calls another constructor). If you declare any constructor, the compiler won't generate a default one automatically.

Q23. What is a destructor?

[OOP]

A destructor releases resources when an object goes out of scope or is deleted. It has no return type, no parameters, and cannot be overloaded. In a class hierarchy, make the base destructor virtual to ensure proper cleanup of derived objects.

Q24. What is a copy constructor? When is it called?

[OOP]

It creates a new object as a copy of an existing one. Called when: (1) object is initialised with another, (2) passed by value to a function, (3) returned by value. Without a custom copy constructor, the compiler generates one that does shallow copy.

```
class A {  
    A(const A& other) { /* deep copy */ }  
};
```

Q25. Explain the Rule of Three / Rule of Five.

[OOP | Modern C++]

Rule of Three: if you define a destructor, copy constructor, or copy assignment, define all three (manual resource management). Rule of Five (C++11): add move constructor and move assignment. Rule of Zero: prefer smart pointers/RAII so you define none of them.

Q26. What is inheritance? What are its types?

[OOP]

Inheritance lets a derived class reuse and extend a base class. Types: Single (one base), Multiple (multiple bases), Multilevel (A->B->C), Hierarchical (one base, many derived), Hybrid (combination). C++ supports all; multiple inheritance can cause the diamond problem.

Q27. What is the diamond problem and how is virtual inheritance solved?

[OOP | Advanced]

When two base classes share a common ancestor and a class inherits from both, the common ancestor's members are duplicated. Virtual inheritance ('class B : virtual public A') ensures only one shared copy of A exists.

```
class A {};  
class B : virtual public A {};  
class C : virtual public A {};  
class D : public B, public C {}; // only one A
```

Q28. What is a virtual function?

[OOP | Polymorphism]

A virtual function allows runtime polymorphism. The correct function is selected based on the actual object type, not the pointer/reference type. The mechanism uses a vtable (virtual function table). Declare with 'virtual' in the base class.

Q29. What is a pure virtual function and an abstract class?

[OOP]

A pure virtual function has no implementation in the base class: 'virtual void f() = 0;'. A class with at least one pure virtual function is abstract and cannot be instantiated. Derived classes must implement all pure virtuals or remain abstract.

Q30. What is the difference between function overriding and overloading?

[OOP]

Overloading: same name, different signature in the same scope (compile-time). Overriding: same name, same signature, redefined in a derived class (runtime polymorphism with virtual). Use 'override' keyword in C++11 to catch errors.

Q31. What does the 'override' keyword do?

[Modern C++]

'override' tells the compiler that a function is intended to override a virtual function. If the base class function doesn't exist or signatures differ, the compiler emits an error. Prevents subtle bugs from signature mismatches.

```
struct Base { virtual void f(int); };  
struct Derived : Base { void f(int) override; }; // OK
```

Q32. What does 'final' keyword do?

[Modern C++]

'final' on a class prevents further inheritance. 'final' on a virtual function prevents further overriding. Allows the compiler to devirtualise calls (performance benefit).

Q33. What is a friend function/class?

[OOP]

A friend has access to private and protected members of the class granting friendship. It is declared inside the class with the 'friend' keyword but defined outside. Friendship is not inherited or transitive.

Q34. What is a static member in a class?

[OOP]

A static data member is shared across all instances — only one copy exists. A static member function has no 'this' pointer and can only access static members. Static members are initialised outside the class definition.

Q35. What is object slicing?

[OOP | Pitfall]

When a derived object is assigned to a base object (by value), the derived-specific data is 'sliced off'. Only the base portion is copied. Prevent by using pointers or references to base, enabling proper polymorphism.

Q36. What is a copy assignment operator?

[OOP]

Allows assignment between objects of the same class (a = b). The compiler generates one (shallow) if not defined. For classes managing resources, implement it with self-assignment check and deep copy. Signature: 'T& operator=(const T& rhs)'.

Q37. What is a conversion constructor?

[OOP]

A constructor taking a single argument that can convert another type to the class type. To prevent implicit conversion use the 'explicit' keyword. Implicit conversions can cause surprising and hard-to-debug behaviour.

```
class A {  
    explicit A(int x) {}  
};  
A a = 5; // Error: explicit prevents implicit conversion
```

Q38. What is the 'this' pointer?

[OOP]

A hidden pointer in every non-static member function pointing to the calling object. Used to resolve ambiguity between member variables and parameters with the same name, and to enable method chaining by returning '*this'.

Q39. What is an initialiser list in constructors?

[OOP | Performance]

Syntax: 'Class::Class(int x) : member(x) {}'. Members are initialised before the constructor body runs. Required for const members, reference members, and base class initialisation. More efficient than assignment inside the body.

Q40. What is polymorphism? Explain its types.

[OOP | Polymorphism]

Polymorphism means 'many forms'. Compile-time (static): function overloading and templates — resolved at compile time. Runtime (dynamic): virtual functions — resolved at runtime via vtable lookup. Runtime polymorphism requires a pointer/reference to base class.

SECTION 3: Memory Management & Pointers

Q41. What is the difference between stack and heap memory?

[Memory]

Stack: fast, automatically managed, LIFO, limited size (~1-8 MB). Used for local variables and function call frames. Heap: large, manually managed (or via smart pointers), slower allocation. Used for dynamic allocation with 'new'/'delete'.

Q42. What is a memory leak? How do you prevent it?

[Memory | Pitfall]

A memory leak occurs when dynamically allocated memory is never freed, causing growing memory usage. Prevent by always pairing new/delete, using RAII, smart pointers (unique_ptr, shared_ptr), and tools like Valgrind or AddressSanitizer.

Q43. What is a dangling pointer?

[Memory | Pitfall]

A pointer that references memory that has already been freed or gone out of scope. Dereferencing it is undefined behaviour. Prevent by setting pointers to nullptr after delete, preferring smart pointers.

```
int* p = new int(5);  
delete p;  
p = nullptr; // safe practice
```

Q44. What is RAII?

[Memory | RAII]

Resource Acquisition Is Initialisation — a C++ idiom where resources are acquired in a constructor and released in the destructor. Guarantees cleanup even if exceptions occur. Smart pointers, file streams, and mutexes follow RAII.

Q45. What is a smart pointer? Name the types.

[Memory | Modern C++]

Smart pointers are wrappers that manage object lifetime automatically. Types: `unique_ptr` (sole ownership, non-copyable), `shared_ptr` (shared ownership via reference counting), `weak_ptr` (non-owning observer to break `shared_ptr` cycles). Defined in `<memory>`.

Q46. What is the difference between `unique_ptr` and `shared_ptr`?

[Memory | Smart Pointers]

`unique_ptr` has exclusive ownership — cannot be copied, only moved. When it goes out of scope, the object is deleted. `shared_ptr` uses reference counting; the object is deleted when the last `shared_ptr` is destroyed. `shared_ptr` has overhead (atomic ref count).

Q47. What is a `weak_ptr` and when should you use it?

[Memory | Smart Pointers]

`weak_ptr` is a non-owning smart pointer that observes a `shared_ptr`-managed object without incrementing the reference count. Use it to break circular references (e.g., parent-child with `shared_ptr` on both sides would be a cycle). Must `lock()` before use.

Q48. What is placement new?

[Memory | Advanced]

Placement new constructs an object at a pre-allocated memory location. Useful for memory pools, embedded systems, and avoiding allocation overhead. You must explicitly call the destructor; do not use `delete`.

```
char buf[sizeof(int)];
int* p = new(buf) int(42); // placement new
p->~int(); // explicit destructor
```

Q49. What is a void pointer?

[Memory]

A `void*` can hold the address of any data type but cannot be dereferenced directly — it must be cast first. Used in C APIs for generic programming. In C++, prefer templates over `void*` for type safety.

Q50. What is a null pointer? What is `nullptr`?

[Memory | Modern C++]

A null pointer does not point to any valid memory location. In C++11+, use `nullptr` (typed null pointer constant of type `nullptr_t`) instead of `NULL` (which is typically 0, causing ambiguity in overloaded functions).

Q51. What is pointer arithmetic?

[Memory]

Adding/subtracting an integer from a pointer moves it by that many elements of the pointed-to type. Valid only within arrays. `p+1` for `int*` advances by 4 bytes (on 32-bit int platforms). Undefined behaviour outside array bounds.

Q52. What is an array and how does it relate to pointers?

[Memory]

An array is a contiguous block of memory. The array name decays to a pointer to the first element in most contexts. However, `sizeof(array)` gives the full size, not just a pointer size. Arrays are not assignable; pointers are.

Q53. Explain memory layout of a C++ program.

[Memory | Systems]

Text segment (code), Data segment (initialised globals/static), BSS segment (uninitialised globals/static), Heap (dynamic allocation, grows up), Stack (function frames, grows down). Understanding this is critical for systems programming.

Q54. What is alignment and padding in structs?

[Memory | Systems]

The compiler adds padding bytes between struct members to align them to their natural alignment (e.g., int on a 4-byte boundary). This can waste space. Use `__attribute__((packed))` or `#pragma pack` to reduce padding (may hurt performance).

Q55. What is a memory pool?

[Memory | Performance]

A memory pool pre-allocates a large block of memory and dispenses chunks from it, avoiding frequent heap allocations. Reduces fragmentation and allocation overhead. Common in game engines, embedded systems, and high-frequency trading.

SECTION 4: STL & Templates

Q56. What is the STL?

[STL]

The Standard Template Library provides generic containers (vector, list, map, set, unordered_map), iterators, algorithms (sort, find, transform), and function objects. It separates data structure from algorithm via the iterator abstraction.

Q57. What is the difference between vector and list?

[STL]

vector: contiguous memory, $O(1)$ random access, $O(n)$ insert/delete in middle, cache-friendly. list: doubly-linked, $O(1)$ insert/delete anywhere, $O(n)$ access, poor cache locality. Prefer vector unless frequent mid-insertions are needed.

Q58. What is the difference between map and unordered_map?

[STL]

map: red-black tree, keys sorted, $O(\log n)$ operations. unordered_map: hash table, $O(1)$ average operations, $O(n)$ worst case, no key ordering. Use unordered_map for speed; map when order matters or keys lack a good hash.

Q59. What are iterators in C++?

[STL]

Iterators abstract the traversal of containers. Types: input, output, forward, bidirectional, random-access. They provide a uniform interface for algorithms. Algorithms work on iterator ranges [begin, end).

Q60. What is a template? What is template specialisation?

[Templates]

A template lets you write generic code parameterised by type or value. Full specialisation provides a specific implementation for a particular type. Partial specialisation works for a subset of template parameters (only for class templates).

```
template
T max(T a, T b) { return a > b ? a : b; }
// Full specialisation:
template<> const char* max(const char* a, const char* b);
```

Q61. What is template metaprogramming (TMP)?

[Templates | Advanced]

TMP uses templates to perform computations at compile time. Classic example: compile-time factorial. Modern C++ uses constexpr instead for readability. TMP enables zero-cost abstractions, type traits, and policy-based design.

```
template struct Factorial { static const int value = N * Factorial::value; };
template<> struct Factorial<0> { static const int value = 1; };
```

Q62. What are variadic templates?

[Templates | Modern C++]

Templates that accept a variable number of arguments (parameter pack). Used to implement std::tuple, std::make_unique, printf-safe functions. Expanded with '...' (ellipsis) and recursion or fold expressions (C++17).

```
template
void print(Args... args) { (std::cout << ... << args) << '\n'; }
```

Q63. What is SFINAE?

[Templates | Advanced]

Substitution Failure Is Not An Error — when template substitution fails for a type, it is removed from overload candidates rather than causing a compile error. Used with std::enable_if to conditionally enable functions based on type traits.

Q64. What is the difference between std::sort and std::stable_sort?

[STL | Algorithms]

std::sort is not stable — equal elements may change relative order; $O(n \log n)$. std::stable_sort preserves relative order of equal elements; $O(n \log n)$ with $O(n)$ extra space or $O(n \log^2 n)$ without.

Q65. What is a lambda expression?

[Modern C++ | STL]

An anonymous function object. Captures variables from enclosing scope (by value [=] or reference [&]). Used with algorithms, callbacks, and async code. Under the hood, the compiler generates a closure class.

```
auto add = [](int a, int b) { return a + b; };
std::sort(v.begin(), v.end(), [](int a, int b){ return a > b; });
```

Q66. What is `std::move`?

[Modern C++ | Performance]

`std::move` casts a value to an rvalue reference, enabling move semantics. It doesn't actually move anything — it allows the move constructor/assignment to be called instead of copy, avoiding expensive deep copies.

Q67. What is `std::forward` and perfect forwarding?

[Modern C++ | Templates]

`std::forward` preserves the value category (lvalue/rvalue) of an argument when passing it to another function. Used in template functions with forwarding references (T&&) to implement perfect forwarding, enabling optimal efficiency.

```
template
unique_ptr make(Args&&... args) {
return unique_ptr(new T(std::forward(args)...));
}
```

Q68. What is a functor (function object)?

[STL | Functional]

A class that overloads `operator()` to behave like a function. Functors can carry state, unlike plain function pointers. Used as comparators, predicates, and callbacks. `std::function` provides a type-erased wrapper for any callable.

Q69. What is range-based for loop?

[Modern C++]

A cleaner syntax for iterating over a container or any type providing `begin()/end()`. Internally calls `begin()` and `end()` and advances with the iterator.

```
for (auto& elem : myVector) { std::cout << elem; }
```

Q70. What is `std::optional` and `std::variant`?

[Modern C++ | C++17]

`optional` represents a value that may or may not be present — a safer alternative to returning sentinel values or pointers. `variant` is a type-safe union holding one of several types. Both introduced in C++17.

SECTION 5: Concurrency & Multithreading

Q71. What is a thread? How do you create one in C++11?

[Concurrency]

A thread is the smallest unit of execution. C++11 introduced `std::thread`. Pass a callable (function, lambda, functor) to the constructor. Call `join()` to wait for completion or `detach()` to let it run independently.

```
std::thread t([]{ std::cout << "Hello from thread\n"; });
t.join();
```

Q72. What is a race condition?

[Concurrency | Pitfall]

A race condition occurs when two or more threads access shared data concurrently and at least one access is a write, with no synchronisation. The outcome depends on thread scheduling — unpredictable

and often incorrect. Detected with thread sanitizer (TSan).

Q73. What is a mutex?

[Concurrency]

A mutex (mutual exclusion object) allows only one thread to access a critical section at a time. Lock with `lock()`, release with `unlock()`. Always prefer `std::lock_guard` or `std::unique_lock` (RAII wrappers) to avoid forgetting to unlock.

```
std::mutex mtx;
{ std::lock_guard lg(mtx); shared_data++; }
```

Q74. What is a deadlock and how do you prevent it?

[Concurrency | Pitfall]

Deadlock occurs when two or more threads wait indefinitely for each other's locks. Prevention: always acquire locks in a consistent global order, use `std::lock()` to lock multiple mutexes atomically, use `timed_mutex` with `try_lock`, minimise lock scope.

Q75. What is a condition variable?

[Concurrency]

A synchronisation primitive that lets threads wait until a condition becomes true. Used with a mutex. `wait()` atomically releases the lock and sleeps; `notify_one()/notify_all()` wakes waiting threads. Always use with a predicate to handle spurious wakeups.

```
std::condition_variable cv;
cv.wait(lock, []{ return data_ready; });
cv.notify_one();
```

Q76. What is `std::atomic`?

[Concurrency | Performance]

`std::atomic` provides lock-free atomic operations on fundamental types. Operations like `fetch_add`, `compare_exchange_strong` are guaranteed to be indivisible. Avoids the overhead of a mutex for simple integer counters or flags.

Q77. What is a thread pool?

[Concurrency | Systems]

A thread pool pre-creates a fixed number of threads and assigns tasks from a queue. Avoids the overhead of creating/destroying threads for each task. Common in server backends. C++20 adds `std::jthread`; custom pools use mutex + condition_variable + queue.

Q78. What is the difference between process and thread?

[Concurrency | Systems]

A process has its own memory space (virtual address space, file descriptors, etc.). Threads within a process share memory and resources but have their own stack and registers. Thread context switches are cheaper than process switches.

Q79. What is memory ordering in C++11?

[Concurrency | Advanced]

Atomic operations can specify memory ordering: relaxed (no sync guarantees), acquire/release (establish happens-before), `seq_cst` (total order, default, most expensive). Choosing weaker ordering can improve performance on multicore architectures.

Q80. What is `std::future` and `std::promise`?

[Concurrency | Modern C++]

promise/future pair: promise sets a value in one thread; future retrieves it in another (blocking if not ready). `std::async` runs a function asynchronously and returns a future. Provides a clean way to communicate results between threads.

```
std::promise p; auto f = p.get_future();
std::thread([&p;]{ p.set_value(42); }).detach();
std::cout << f.get();
```

Q81. What is false sharing?

[Concurrency | Performance]

False sharing occurs when two threads modify variables on the same cache line, causing constant cache invalidation between CPU cores — a significant performance bottleneck. Fix by padding data to separate cache lines (`alignas(64)`).

Q82. What is a semaphore?

[Concurrency]

A semaphore controls access to a limited number of resources. Binary semaphore (0 or 1) is similar to a mutex. Counting semaphore allows N threads concurrently. C++20 adds `std::counting_semaphore` and `std::binary_semaphore`.

Q83. What is the reader-writer problem?

[Concurrency | Systems]

Multiple readers can access data simultaneously, but writers need exclusive access. `std::shared_mutex` (C++17) allows `std::shared_lock` (readers) and `std::unique_lock` (writers), implementing read-write locking efficiently.

Q84. What is lock-free programming?

[Concurrency | Advanced]

Algorithms that use atomic operations instead of locks to avoid blocking. Typically use compare-and-swap (CAS) loops. Pros: no deadlocks, better scalability. Cons: complex to reason about, ABA problem, livelock possible.

Q85. What is coroutine in C++20?

[Concurrency | C++20]

Coroutines are functions that can be suspended and resumed. They enable cooperative multitasking and asynchronous programming without threads. Defined with `co_await`, `co_yield`, `co_return`. The compiler transforms them into state machines.

SECTION 6: Modern C++ (C++11/14/17/20)

Q86. What are rvalue references and move semantics?

[Modern C++ | Performance]

Rvalue references (T&&) bind to temporaries and rvalues. Move semantics transfer ownership of resources instead of copying. Dramatically reduces copies when returning large objects or inserting temporaries into containers.

```
std::vector src = {1,2,3};
```

```
std::vector dst = std::move(src); // src is now empty
```

Q87. What is auto keyword?

[Modern C++]

auto instructs the compiler to deduce the variable type from the initialiser. Reduces verbosity, prevents implicit conversions, and adapts automatically to type changes. Use with care — explicit types sometimes improve readability.

Q88. What is decltype?

[Modern C++]

decltype(expr) yields the type of an expression without evaluating it. Used in templates and trailing return types to express types that depend on arguments.

```
auto f(int a, double b) -> decltype(a + b) { return a + b; }
```

Q89. What is constexpr?

[Modern C++]

constexpr specifies that a function or variable can be evaluated at compile time. More powerful than const; allows functions with logic. Enables zero-overhead abstractions. In C++14+ constexpr functions can have loops and conditions.

```
constexpr int factorial(int n) { return n <= 1 ? 1 : n * factorial(n-1); }  
constexpr int f5 = factorial(5); // computed at compile time
```

Q90. What is structured binding (C++17)?

[C++17]

Structured bindings decompose objects into named components. Works with pairs, tuples, arrays, and aggregates. Makes code cleaner, especially with range-based for loops over maps.

```
auto [key, value] = *myMap.find("x");  
auto [a, b, c] = std::tuple{1, 2.0, '3'};
```

Q91. What are fold expressions (C++17)?

[C++17 | Templates]

Fold expressions reduce a parameter pack using a binary operator. Eliminates recursive template helpers for variadic operations. Syntax: (args op ...) for right fold, (... op args) for left fold.

```
template  
auto sum(T... args) { return (... + args); }
```

Q92. What is if constexpr (C++17)?

[C++17 | Templates]

if constexpr evaluates the condition at compile time and discards the dead branch entirely. Simplifies template metaprogramming — avoids SFINAE for branch-based dispatch.

```
template  
void print(T v) {  
    if constexpr (std::is_integral_v) std::cout << "int: " << v;  
    else std::cout << "other: " << v;  
}
```

Q93. What are type traits?

[Templates | Modern C++]

Type traits (in `<type_traits>`) provide compile-time information about types: `is_integral`, `is_pointer`, `is_same`, `remove_const`, `conditional`, etc. Foundation for SFINAE and `constexpr` branching in generic code.

Q94. What is `std::string_view` (C++17)?

[C++17 | Performance]

`string_view` is a non-owning, read-only view of a string (or substring). Avoids copies when passing strings to functions. More efficient than `const std::string&` for string literals. Be careful about lifetime — the underlying string must outlive the view.

Q95. What are `std::span` (C++20)?

[C++20]

`span` is a non-owning view over a contiguous sequence (array, vector, raw pointer + size). Replaces (`T* ptr`, `size_t n`) pairs in API design. Provides bounds-checked access and interoperability.

Q96. What is `std::any` (C++17)?

[C++17]

`std::any` holds a value of any copyable type with type-erased storage. Access the value with `std::any_cast`. Throws `std::bad_any_cast` if the type doesn't match. Less safe than `variant`; use `variant` when types are known at compile time.

Q97. What are concepts (C++20)?

[C++20 | Templates]

Concepts are named compile-time predicates on template parameters. They replace and improve upon SFINAE with clear, readable constraints. Produce much better error messages. Defined with 'concept' keyword and used with 'requires' clause.

```
template
concept Numeric = std::is_arithmetic_v;
template T add(T a, T b) { return a + b; }
```

Q98. What are ranges (C++20)?

[C++20 | STL]

The Ranges library (`std::ranges`) provides composable, lazy view-based algorithms. Ranges are sequences with `begin/end`. Views are lazy transformations (`filter`, `transform`, `take`). Enables pipeline-style code with the pipe `|` operator.

```
auto result = v | std::views::filter([](int x){ return x%2==0; })
| std::views::transform([](int x){ return x*x; });
```

Q99. What is three-way comparison (spaceship operator, C++20)?

[C++20]

The `<=>` (spaceship) operator returns `std::strong_ordering`, `std::weak_ordering`, or `std::partial_ordering`. Defining it for a class auto-generates all six comparison operators (`==`, `!=`, `<`, `<=`, `>`, `>=`). Reduces boilerplate significantly.

```
auto operator<=>(const MyClass&) const = default;
```

Q100. What is `std::jthread` (C++20)?

[C++20 | Concurrency]

`std::jthread` is like `std::thread` but automatically joins on destruction (RAII). Also supports cooperative cancellation via `std::stop_token`. Eliminates the most common `std::thread` pitfall (forgetting to join).

SECTION 7: Backend Development Fundamentals

Q101. What is REST? What are its constraints?

[Backend | API]

REST (Representational State Transfer) is an architectural style for distributed hypermedia systems. Constraints: client-server, statelessness, cacheability, uniform interface (URIs, HTTP verbs, HATEOAS), layered system, code on demand (optional).

Q102. What is the difference between REST and gRPC?

[Backend | API]

REST uses HTTP/1.1 + JSON (text), is widely supported, easy to use from browsers. gRPC uses HTTP/2 + Protocol Buffers (binary), is faster and strongly typed, supports streaming, ideal for microservice-to-microservice communication.

Q103. What is an HTTP request/response lifecycle?

[Backend | Networking]

Client resolves DNS -> TCP/TLS handshake -> sends HTTP request (method, headers, body) -> server processes -> returns response (status code, headers, body) -> connection kept alive (keep-alive) or closed.

Q104. What is the difference between HTTP/1.1, HTTP/2, and HTTP/3?

[Backend | Networking]

HTTP/1.1: text-based, one request per connection (with pipelining issues). HTTP/2: binary, multiplexed (multiple requests on one TCP connection), header compression (HPACK), server push. HTTP/3: uses QUIC (UDP-based), eliminates TCP head-of-line blocking.

Q105. What is a database index?

[Backend | Database]

An index is a data structure (B-tree or hash) that speeds up data retrieval. Reduces full table scans. Trade-off: faster reads, slower writes, extra storage. Primary key is automatically indexed. Add indexes on frequently queried/filtered columns.

Q106. What is the difference between SQL and NoSQL?

[Backend | Database]

SQL: relational, ACID compliant, structured schema, good for complex queries and transactions. NoSQL: various models (document, key-value, column, graph), horizontally scalable, schema-flexible, eventual consistency in many cases.

Q107. What are ACID properties?

[Backend | Database]

Atomicity (all or nothing), Consistency (database stays valid), Isolation (concurrent transactions don't interfere), Durability (committed data persists). Relational databases guarantee ACID; many NoSQL stores relax these for scalability.

Q108. What is a transaction?

[Backend | Database]

A transaction is a unit of work that is atomic. Either all statements succeed and are committed, or all are rolled back on failure. Managed with BEGIN/COMMIT/ROLLBACK. Isolation levels (Read Uncommitted,

Read Committed, Repeatable Read, Serializable) control concurrency trade-offs.

Q109. What is caching? What are cache strategies?

[Backend | Performance]

Caching stores frequently accessed data in fast storage. Strategies: Cache-aside (app manages), Read-through (cache fills on miss), Write-through (write to cache and DB together), Write-back (write to cache, async to DB). Tools: Redis, Memcached.

Q110. What is a message queue? Name examples.

[Backend | Messaging]

A message queue decouples producers and consumers — producers publish messages; consumers process asynchronously. Enables buffering, load levelling, retry, and fan-out. Examples: RabbitMQ (AMQP), Apache Kafka (distributed log), AWS SQS.

Q111. What is the CAP theorem?

[Backend | Distributed Systems]

A distributed system can guarantee only two of three: Consistency (all nodes see same data), Availability (every request receives a response), Partition Tolerance (system works despite network splits). In practice, partitions are unavoidable, so you choose between CP or AP.

Q112. What is horizontal vs vertical scaling?

[Backend | Scalability]

Vertical scaling: add more CPU/RAM to one machine (scale up). Simple but has limits. Horizontal scaling: add more machines (scale out). Requires distributed architecture, load balancing, stateless services.

Q113. What is a load balancer?

[Backend | Infrastructure]

A load balancer distributes incoming requests across multiple server instances. Algorithms: round-robin, least connections, IP hash, weighted. Types: L4 (TCP/UDP level), L7 (HTTP level — can route by URL/header). Examples: NGINX, HAProxy, AWS ALB.

Q114. What is a CDN?

[Backend | Infrastructure]

A Content Delivery Network distributes static assets (images, JS, CSS) to geographically distributed edge servers close to users. Reduces latency, offloads origin servers, improves availability. Examples: Cloudflare, Akamai, AWS CloudFront.

Q115. What is OAuth 2.0 and JWT?

[Backend | Security]

OAuth 2.0 is an authorisation framework allowing third-party apps to access resources on behalf of a user without sharing credentials. JWT (JSON Web Token) is a compact, self-contained token encoding claims as signed JSON, used for stateless authentication.

Q116. What is SQL injection and how to prevent it?

[Backend | Security]

SQL injection occurs when user input is embedded directly in SQL queries, allowing attackers to manipulate the database. Prevent with parameterised queries/prepared statements, ORMs, input validation, and least privilege database accounts.

```
// Vulnerable: SELECT * FROM users WHERE name='+name+'
// Safe: use prepared statements with ? or $1 placeholders
```

Q117. What is the difference between concurrency and parallelism?

[Backend | Concurrency]

Concurrency: multiple tasks are in progress (possibly on one core, interleaved). Parallelism: multiple tasks run simultaneously on multiple cores/CPU's. Concurrency is about structure; parallelism is about execution.

Q118. What is microservices architecture?

[Backend | Architecture]

Microservices decompose an application into small, independently deployable services, each owning its data. Benefits: independent scaling, fault isolation, polyglot. Challenges: network latency, distributed transactions, operational complexity.

Q119. What is Docker and why is it used in backend development?

[Backend | DevOps]

Docker packages applications and dependencies into portable containers that run consistently across environments. Solves 'works on my machine'. Used for dev/test parity, CI/CD pipelines, and Kubernetes deployments.

Q120. What is WebSocket and when is it used?

[Backend | Networking]

WebSocket provides full-duplex communication over a single TCP connection, enabling real-time bi-directional messaging. Used for live chat, gaming, stock tickers, collaborative editing. Initiated by HTTP upgrade request.

SECTION 8: System Design & Architecture

Q121. How would you design a URL shortener (like bit.ly)?

[System Design]

Key components: API server, hash function (base62 of MD5 or random ID), database mapping short->long URL, cache (Redis) for hot URLs, redirect logic (301 permanent or 302 temporary). For scale: read replicas, CDN, rate limiting per user.

Q122. How would you design a rate limiter?

[System Design]

Algorithms: Token Bucket (smooth bursts), Leaky Bucket (constant output), Fixed Window Counter, Sliding Window Log, Sliding Window Counter. Store counts in Redis (atomic INCR + EXPIRE). Distribute via API gateway or middleware.

Q123. What is consistent hashing?

[System Design | Distributed]

Consistent hashing maps both nodes and keys onto a virtual ring. Adding/removing a node only remaps ~K/n keys (K=keys, n=nodes) instead of all keys. Used in distributed caches (Memcached, DynamoDB) and load balancers.

Q124. What is the difference between synchronous and asynchronous APIs?

[System Design | API]

Synchronous: caller blocks until the operation completes. Simple but can cause latency. Asynchronous: caller continues; result delivered via callback, future, event, or polling. Necessary for I/O-heavy, long-running operations.

Q125. What is event-driven architecture?

[System Design | Architecture]

Components communicate via events published to a bus/queue. Producers emit events; consumers react. Enables loose coupling, horizontal scaling, and audit trails. Core of reactive systems and CQRS/Event Sourcing patterns.

Q126. What is CQRS?

[System Design | Architecture]

Command Query Responsibility Segregation separates write operations (commands that change state) from read operations (queries). Read and write models can be separately optimised and scaled. Often paired with Event Sourcing.

Q127. What is a circuit breaker pattern?

[System Design | Resilience]

A circuit breaker monitors failures when calling an external service. After a threshold, it 'opens' and fast-fails requests (avoiding cascading failures). After a timeout, it enters 'half-open' state to probe recovery. Implemented in Hystrix, Resilience4j.

Q128. What is eventual consistency?

[System Design | Distributed]

In distributed systems, instead of guaranteeing immediate consistency (strong), the system promises that all nodes will eventually converge to the same state given no new updates. Common in NoSQL (DynamoDB, Cassandra). Acceptable for many use cases.

Q129. What is sharding?

[System Design | Database]

Sharding horizontally partitions a database by splitting data across multiple instances by a shard key (e.g., user ID). Each shard holds a subset. Enables massive scale but complicates cross-shard queries and transactions.

Q130. What is a write-ahead log (WAL)?

[System Design | Database]

A WAL records every database change before applying it to data files. On crash, the database replays the WAL to recover committed transactions. Foundation of ACID durability in PostgreSQL, SQLite, and others. Also used in distributed consensus (Raft, Paxos).

Q131. What is an API gateway?

[System Design | Microservices]

An API gateway is a reverse proxy that sits in front of microservices, handling cross-cutting concerns: authentication, authorisation, rate limiting, request routing, SSL termination, logging, and caching. Examples: Kong, AWS API Gateway, NGINX.

Q132. What is Kafka and how is it different from RabbitMQ?

[System Design | Messaging]

Kafka is a distributed log/event streaming platform — messages are retained and replayable. High throughput, consumer groups, partition-based parallelism. RabbitMQ is a traditional message broker with sophisticated routing but messages are typically consumed and deleted.

Q133. What is a distributed transaction? How do you handle it?

[System Design | Distributed]

A transaction spanning multiple services/databases. Approaches: Two-Phase Commit (2PC) — strong consistency but blocking; Saga pattern — sequence of local transactions with compensating transactions on failure; Outbox pattern — transactional outbox with event log.

Q134. How does a B-tree differ from a B+ tree?

[System Design | Database]

B-tree stores data in all nodes. B+ tree stores data only in leaf nodes (internal nodes hold keys for routing); leaves are linked for efficient range scans. B+ trees are preferred in database indexes (MySQL InnoDB, PostgreSQL) for range queries.

Q135. What is observability in backend systems?

[System Design | Operations]

Observability is the ability to understand internal system state from external outputs. Three pillars: Logs (event records), Metrics (quantitative measurements — latency, error rate, throughput), Traces (distributed request path). Tools: ELK, Prometheus+Grafana, Jaeger.

SECTION 9: Product-Based Company Interview Questions

Q136. Implement a thread-safe Singleton in C++11.

[FAANG | Design Pattern]

Use a local static variable (Meyers Singleton). C++11 guarantees that static local initialisation is thread-safe. No need for double-checked locking.

```
class Singleton {
Singleton() = default;
public:
static Singleton& get() {
static Singleton instance;
return instance;
}
Singleton(const Singleton&) = delete;
Singleton& operator=(const Singleton&) = delete;
};
```

Q137. What is the difference between deep copy and shallow copy?

[FAANG | Memory]

Shallow copy copies the pointer (both objects point to the same memory). Deep copy allocates new memory and copies the actual data. Shallow copy causes double-free and dangling pointer bugs when one object is destroyed.

Q138. Implement a LRU Cache in C++.

[FAANG | Data Structures]

Use a doubly-linked list (for O(1) insert/delete) and an unordered_map (for O(1) lookup). On access, move the node to the front. On insert beyond capacity, remove from the tail.

```
// unordered_map<::iterator> cache;
// list< lru; // {key, value}
// get: if found, splice to front; put: evict tail if full
```

Q139. What is vtable and how does it work?

[FAANG | C++ Internals]

The compiler generates a vtable (array of function pointers) for each class with virtual functions. Each object has a hidden vptr pointing to its class's vtable. Virtual function calls dereference vptr, lookup the function pointer, and call it — one extra indirection.

Q140. What is the output of this code? (tricky C++ question)

[FAANG | Gotcha]

```
class Base { public: virtual void f() { cout<<"Base"; } void g() { cout<<"BaseG"; } }; class D : public Base {
public: void f() { cout<<"Derived"; } void g() { cout<<"DerivedG"; } }; Base* p = new D; p->f(); p->g(); —
Output: 'DerivedBase G'. f() is virtual (runtime dispatch), g() is not (compile-time, uses Base).
```

Q141. What are the costs of using virtual functions?

[FAANG | Performance]

1. vtable storage per class. 2. vptr storage per object (one pointer). 3. One extra indirection per virtual call. 4. Inhibits inlining. 5. Breaks certain hardware optimisations (branch predictor). In hot paths, consider CRTP (Curiously Recurring Template Pattern) for static polymorphism.

Q142. What is CRTP (Curiously Recurring Template Pattern)?

[FAANG | Advanced | Templates]

CRTP provides compile-time polymorphism without vtable overhead. The base class is templated on the derived class, allowing static dispatch. Used in policy-based design, mixin classes, and performance-critical code.

```
template
struct Base {
void interface() { static_cast(this)->impl(); }
};
struct MyClass : Base { void impl() { /*...*/ } };
```

Q143. Explain the output of this: int a=1,b=2; cout << a++ << b++;

[FAANG | Gotcha]

Output is '12'. In cout, both a++ and b++ use the original values (post-increment returns old value). The expressions are evaluated before printing. After the statement, a=2 and b=3.

Q144. What is undefined behaviour (UB) in C++? Give examples.

[FAANG | C++ Internals]

UB is code whose behaviour is not defined by the C++ standard — the compiler may do anything. Examples: signed integer overflow, dereferencing null/dangling pointer, reading uninitialised variable, accessing out-of-bounds array, data race. UB enables aggressive compiler optimisations but is a major source of bugs.

Q145. How would you implement a producer-consumer queue in C++?

[FAANG | Concurrency]

Use a `std::queue` protected by a `std::mutex`. Producer locks, pushes, unlocks, then notifies a `condition_variable`. Consumer waits on the `condition_variable` (with a predicate), locks, pops, processes. Add a bounded size for backpressure.

Q146. What is the difference between `emplace_back` and `push_back`?

[FAANG | STL | Performance]

`push_back` constructs an object and then copies/moves it into the container. `emplace_back` constructs the object in-place using the arguments directly, avoiding an extra copy/move. Prefer `emplace_back` for non-trivially constructible types.

Q147. What is copy elision and RVO (Return Value Optimisation)?

[FAANG | Performance]

Copy elision is a compiler optimisation that eliminates unnecessary copies. Named Return Value Optimisation (NRVO) constructs the return value directly in the caller's stack frame. Mandatory in C++17 for prvalue returns (guaranteed copy elision).

Q148. How does `std::shared_ptr` implement reference counting?

[FAANG | Memory | Smart Pointers]

`shared_ptr` stores two pointers: one to the managed object and one to a control block (shared reference count, weak reference count, deleter). The count is atomic for thread safety. When the shared count reaches 0, the object is destroyed; when the weak count reaches 0, the control block is freed.

Q149. Design a parking lot system in C++.

[FAANG | OOD]

Classes: `ParkingLot`, `Floor`, `Slot` (Small/Medium/Large), `Vehicle` (Bike/Car/Truck), `Ticket`. `ParkingLot::park()` finds the nearest available matching slot, creates a ticket. `unpark()` calculates fee based on duration and slot type. Use OOP, inheritance for `Vehicle` types, strategy pattern for fee calculation.

Q150. What are common C++ interview mistakes to avoid?

[FAANG | Best Practices]

1. Forgetting to make destructors virtual in base classes. 2. Returning references to local variables. 3. Using C-style casts instead of C++ casts. 4. Ignoring RAI — manual `new/delete`. 5. Off-by-one in pointer arithmetic. 6. Not handling self-assignment in `operator=`. 7. Assuming operator evaluation order.

Q151. What is the difference between `std::endl` and `'\n'`?

[FAANG | Performance]

Both output a newline. `std::endl` also flushes the output buffer — significantly slower in a loop. Prefer `'\n'` for performance. Use `std::endl` only when you explicitly need to flush (e.g., before a program waits for input).

Q152. What is a fold expression and how is it better than recursion?

[FAANG | C++17 | Templates]

C++17 fold expressions reduce a parameter pack with an operator in a single expression — no recursion, no helper struct, direct compiler handling. Cleaner, shorter, and compiles faster than the equivalent recursive TMP.

Q153. What is the Pimpl idiom?

[FAANG | Design Pattern]

Pointer to Implementation hides private implementation details in a separate class (accessed via a pointer). Reduces compilation dependencies, allows ABI stability, and hides implementation. Used in library APIs. Pairs with `unique_ptr` for the impl pointer.

```
// widget.h:  
class Widget {  
    struct Impl; std::unique_ptr pImpl;  
public: void draw();  
};  
// widget.cpp defines Impl and all methods
```

Q154. How do you handle exceptions safely in C++?

[FAANG | Exceptions]

Prefer RAII so resources are freed on exception. Use `noexcept` where appropriate (move constructors should be `noexcept`). Catch by reference (`catch(const std::exception& e)`). Never let exceptions escape destructors. Use exception hierarchies for structured error handling.

Q155. What is the difference between a process's heap and the free store?

[FAANG | Memory | Advanced]

The free store is the C++ abstraction for memory allocated with `new/delete` — managed by the allocator. The heap is the OS-level memory region. In practice they refer to the same memory, but conceptually the free store allows custom allocators that may source memory differently (e.g., from a pool).

All 155 Questions Covered!

C++ Fundamentals | OOP | Memory & Pointers | STL & Templates | Concurrency
Modern C++11/14/17/20 | Backend Development | System Design | FAANG Specials

Best of luck with your interviews!